School Of Computer Science Engineering And Information System

M.TECH SOFTWARE ENGINEERING

WINTER SEMESTER 2024-25

SWE4010 –ARTIFICIAL INTELLIGENCE

SLOT – E1+TE1

J COMPONENT

REVIEW 3

TITLE: "DOCUMENT PARSER"

UNDER GUIDANCE OF

Dr. Ganesan K

SUBMITTED BY

Poojasree S - 22MIS0251

Swathi V - 22MIS0613

Document Parser

Abstract:

In today's digital era, extracting meaningful information from various document formats is crucial for automation and efficiency. The AI Document Parser is an intelligent system designed to process and extract structured data from documents such as PDFs, Word files, and scanned images. Utilizing Natural Language Processing (NLP), Optical Character Recognition (OCR), and, the parser identifies key entities, extracts text, and converts unstructured data into a structured format. This tool is particularly useful for lawyers, legal advisors, and real estate professionals dealing with legal documents related to land and property, such as ownership deeds, agreements, and contracts. It can recognize legal clauses, extract property details, and provide quick insights, reducing manual effort and improving document analysis efficiency. By automating document processing, this AI-powered tool enhances accuracy and streamlines workflows, making it a valuable asset for various industries.

Introduction:

With the rapid digitization of information, the ability to extract structured data from various document formats has become increasingly important. The AI Document Parser is a smart, Python-based system designed to automate this process using Natural Language Processing (NLP) and Optical Character Recognition (OCR). It can handle PDFs, Word files, and scanned images to extract key entities and convert unstructured content into a structured format.

This tool is especially beneficial for legal and real estate professionals who work with documents like ownership deeds, contracts, and agreements. By recognizing legal clauses, property information, and other critical details, it significantly reduces manual work, improves accuracy, and streamlines document analysis workflows.

Problem statement:

Advocates and legal professionals often deal with large volumes of complex documents such as case files, agreements, property deeds, and legal notices. Manually reading and extracting relevant information from these documents is time-consuming, error-prone, and inefficient—especially when working under tight deadlines or handling multiple cases simultaneously.

There is a growing need for an intelligent solution that can automatically process these documents, identify key legal entities such as party names, advocate details, case numbers, dates, and clauses, and present the extracted data in a structured format.

The problem lies in the unstructured nature of most legal documents, which vary in format and language, making standardization difficult. Advocates require a tool that can reduce manual workload, improve accuracy, and streamline case preparation and document review.

This project aims to address this problem by developing an AI-powered Document Parser tailored for advocates, capable of handling diverse legal documents and extracting critical information using technologies like NLP, OCR, and Python automation.

Solution: AI-Powered Legal Document Parser for Advocates

To address the challenges faced by legal professionals in processing unstructured legal documents, this project proposes the development of an AI-powered Legal Document Parser. The system will leverage Natural Language Processing (NLP), Optical Character Recognition (OCR), and Python automation to intelligently extract, organize, and present key information from legal texts.

Core Features:

1.Automatic Text Extraction

pdfplumber extracts text from standard PDFs.

OCR fallback (using pytesseract) processes scanned or image-based PDFs.

Ensures text extraction reliability from any kind of legal document.

2. Intelligent Information Parsing

Extracts key legal details using pattern recognition and NLP:

Court Name

Case Number

Plaintiff & Defendant

Filing Date & Promissory Note Date

Principal Amount, Interest Amount, Total Claim

Advocate Name

Advocate Address

3. Structured Output Presentation

Cleanly formatted display of extracted data.

Labels missing or unidentifiable fields as "Not found" to ensure clarity.

Easy to review and validate.

4. PDF Upload Support

Users can upload PDF documents interactively using Google Colab.

Supports single-click upload and instant processing.

5. Dual Processing Approach

Combines text-based parsing and image-based OCR.

Automatically switches to OCR when text is unreadable or missing.

Handles both typed and scanned legal documents.

6. Error Handling

Graceful handling of file reading and parsing errors.

Informs user of fallback usage (e.g., OCR) for better transparency.

7. Modular Design

Separate functions for text extraction, parsing, displaying results, and overall processing.

Easy to update or add more extraction rules in future.

Modules and Their Functionality

1. Text Extraction Module

Function: extract_text_from_pdf(pdf_path)

Uses pdfplumber to extract text from PDF.

If extraction fails or text is too short, uses OCR (pytesseract) to extract text from images of the PDF.

Converts PDF pages into images using pdf2image for OCR fallback.

Handles both scanned and native PDFs.

2. Legal Entity Parsing Module

Function: parse_legal_document(text)

Uses regex patterns and NLP (spaCy) to extract legal information from unstructured text.

3. Output Display Module

Function: display_legal_info(info)

Neatly prints extracted information with labels.

Handles missing fields by showing Not found.

4. Main Flow

Function: process_legal_document()

Allows PDF upload via Google Colab

Calls all other modules:

Uploads file

Extracts and parses text

Displays structured information

Benefits:

Saves Time: Automates reading and parsing of complex documents

Reduces Errors: Minimizes manual extraction mistakes

Boosts Productivity: Advocates can focus on analysis, not formatting

Scalable: Can be expanded to support more document types and languages

Possible Enhancements

Improve Accuracy with NLP Models

Train or fine-tune a Legal BERT model for better NER (e.g., extracting sections like “Judgment,” “Relief,” etc.)

GUI-based Interface

Use Streamlit or a web-based dashboard for non-technical users.

Save Output to JSON/CSV

Enable download of extracted data in structured file formats.

Support for Regional Languages

Integrate translation APIs or multilingual OCR.

Document Classification

Automatically detect the type of document (petition, judgment, etc.).

Implementation:

```
!pip install pdf2image # Install pdf2image to convert PDF to images
!pip install pytesseract
!pip install pdfplumber # Install the pdfplumber library

#  Import Libraries
import pytesseract
import pdfplumber
import re
import spacy
from pathlib import Path
from pdf2image import convert_from_path
from google.colab import files
from PIL import Image
#  Load NLP Model
nlp = spacy.load("en_core_web_sm")

#  Extract text using OCR fallback
def extract_text_from_pdf(pdf_path):
    text = ""
    try:
        with pdfplumber.open(pdf_path) as pdf:
            for page in pdf.pages:
                # Indented block for the 'for' loop
                page_text = page.extract_text()
                if page_text:
```

```
                text += page_text + "\n"
        # Fallback to OCR if no text found or text is too short
        if len(text.strip()) < 100:
            print("Using OCR fallback...")
            images = convert_from_path(pdf_path)
            for i, img in enumerate(images):
                img_text = pytesseract.image_to_string(img)
                text += img_text + "\n"
                print(f"Processed page {i+1} with OCR")
    except Exception as e:
        print(f"Error extracting text: {e}")
    return text# Extract Legal Info from document content


def parse_legal_document(text):
    info = {}


    # Court Name
    court_match = re.search(r'In the Court of the\s+(.*?)\s+at\s+([A-Za-z]+)',
text, re.IGNORECASE)
    if court_match:
        info['court_name'] = f"{court_match.group(1)} at {court_match.group(2)}"


    # Case Number
    case_match = re.search(r'O\.S\.?\s*No\.?\s*[:\-]?\s*(\d+)', text,
re.IGNORECASE)
    if case_match:
        info['case_number'] = case_match.group(1)
```

```
    # Plaintiff & Defendant
    parties = re.findall(r'([A-Z][a-z]+\s+S/o\s+[A-Z][a-
z]+).*?\b(Plaintiff|Defendant)\b', text)
    for name, role in parties:
        info[role.lower()] = name.strip()

    # Dates (Promissory note and Filing)
    dates = re.findall(r'\b\d{1,2}[./-]\d{1,2}[./-]\d{2,4}\b', text)
    if len(dates) >= 1:
        info['filing_date'] = dates[0]
    if len(dates) >= 2:
        info['promissory_note_date'] = dates[1]

    # Principal Amount (Rupees section or table style)
    principal_match = re.search(r'Principal amount.*?Rs\.?\s?([\d,]+)', text,
re.IGNORECASE)
    if not principal_match:
        principal_match = re.search(r'Rs\.?\s?(0)', text)
    if principal_match:
        info['principal_amount'] = principal_match.group(1).replace(",", "")

    # Interest Amount
    interest_match = re.search(r'towards interest.*?Rs\.?\s?([\d,]+)', text,
re.IGNORECASE)
    if not interest_match:
        interest_match = re.search(r'Rs\.?\s?(0)', text)
```

```
    if interest_match:
        info['interest_amount'] = interest_match.group(1).replace(",", "")

    # Total Claim
    total_match = re.search(r'(aggregating.*?Rs\.?\s?([\d,]+))', text,
re.IGNORECASE)
    if total_match:
        info['total_claim'] = total_match.group(2).replace(",", "")

    # Advocate Name
    adv_name_match = re.search(r'Address for service of the Plaintiff
is\s+([\s\S]*?)\n', text)
    if adv_name_match:
        advocate_block = adv_name_match.group(1)
        name_match = re.search(r'([A-Z]\.\s?[A-Z][a-z]+(?:\s+[A-Z][a-z]+)?)',
advocate_block)
        if name_match:
            info['advocate'] = name_match.group(1).strip()

    # Advocate Address (Multi-line safe)
    address_block_match = re.search(
    r'Address for service of the Plaintiff is\s*((?:.*\n){1,4})',
    text,
    re.IGNORECASE
    )
    if address_block_match:
        address_lines = address_block_match.group(1).strip().splitlines()
```

```
        # Filter lines that look like address (ignore advocate name lines)
        address_only = [line.strip() for line in address_lines if re.search(r'\d+|Road|Street|Nagar|Village|[A-Za-z]{3,}', line)]
        info['address'] = ", ".join(address_only[-2:])  # Pick last 2 lines as address
    else:
        info['address'] = "Not found"
    return info # This line was moved outside the if block# 📊 Display results neatly
def display_legal_info(info):
    print("\n" + "="*50)
    print("📌 EXTRACTED LEGAL INFORMATION:")
    print("="*50)

    fields = [
        ("Court", 'court_name'),
        ("Case Number", 'case_number'),
        ("Plaintiff", 'plaintiff'),
        ("Defendant", 'defendant'),
        ("Filing Date", 'filing_date'),
        ("Promissory Note Date", 'promissory_note_date'),
        ("Principal Amount", 'principal_amount'),
        ("Interest Amount", 'interest_amount'),
        ("Total Claim", 'total_claim'),
        ("Advocate", 'advocate'),
        ("Address", 'address')
    ]
```

```
    for label, key in fields:
        value = info.get(key)
        if value:
            print(f"{label:20}: {value}")
        else:
            print(f"{label:20}: Not found")

    print("="*50)

# Main function
def process_legal_document():
    # Upload the PDF
    print("Please upload your legal document (PDF format)...")
    uploaded = files.upload()
    file_path = list(uploaded.keys())[0]

    print(f"\nProcessing {file_path}...")
 # Extract text
    document_text = extract_text_from_pdf(file_path)
  # Parse the document
    legal_info = parse_legal_document(document_text)

    # Display the results
    display_legal_info(legal_info)
    return legal_info, document_text
# Run the full process
```

```
if __name__ == "__main__":
    legal_info, text = process_legal_document()
```

Output:

```
==================================================
📌 EXTRACTED LEGAL INFORMATION:
==================================================
Court                : Principal Subordinate Judge at Tindivanam
Case Number          : 2024
Plaintiff            : Thiagarajan S/o Balasundaram
Defendant            : Velmurugan S/o Manjini
Filing Date          : 18.01.2021
Promissory Note Date: 18.01.2021
Principal Amount     : 200000
Interest Amount      : 72000
Total Claim          : 272000
Advocate             : K. Senthil Kumar
Address              : No.36, Pondy Road,, Marakkanam
==================================================
```

Conclusion

This project successfully demonstrates an AI-powered solution for automating the extraction of legal information from complex and unstructured documents. By combining PDF parsing, OCR technology, and NLP techniques, the system effectively identifies and structures essential details such as case numbers, party names, dates, monetary amounts, and advocate details.

This tool significantly reduces manual effort, enhances accuracy, and streamlines the document review process for advocates and legal professionals. With further enhancements, such as support for multiple languages and integration with legal databases, the system holds great potential to become an indispensable assistant in the legal domain.